

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Bacharelado em Engenharia de Software

Programação Modular

DOCUMENTO TÉCNICO

Sprint 4 — Evolução Arquitetural e Padrões de Projeto

Sistema de Hospedagem

Grupo 5

Vinicius Fernandes Soares Cardoso

Daniel Oliveira de Menezes

Belo Horizonte

2025

1. Introdução

Este documento descreve as decisões arquiteturais adotadas na Sprint 4 do Sistema de Hospedagem, desenvolvido como trabalho prático da disciplina de Programação Modular. Nesta etapa, o sistema foi evoluído por meio da aplicação de padrões de projeto clássicos (GoF), com o objetivo de aumentar sua extensibilidade, reduzir o acoplamento entre componentes e reforçar a aderência aos princípios da orientação a objetos, em especial o princípio Aberto/Fechado (OCP).

Foram implementadas duas funcionalidades — a Central de Notificações e a Montagem de Pacotes de Hospedagem —, além do uso obrigatório do padrão Singleton na gestão da central de notificações.

2. Funcionalidades e Padrões Aplicados

2.1 Central de Notificações — Padrão Observer

O sistema passou a notificar automaticamente a ocorrência de eventos relevantes, como a criação e o cancelamento de uma reserva. As notificações são enviadas por múltiplos canais de forma independente: e-mail, SMS, WhatsApp e notificação interna.

O problema central era evitar que o `AluguelService` precisasse conhecer e invocar diretamente cada canal de comunicação, o que geraria alto acoplamento e dificultaria a inclusão de novos canais. O padrão Observer foi adotado para desacoplar o emissor dos eventos (`AluguelService`) dos receptores (canais de notificação): o `GerenciadorDeNotificacoes` mantém uma lista de observadores do tipo `Notificavel` e os notifica automaticamente sempre que um evento é disparado.

Elemento	Tipo	Papel no padrão
Notificavel	Interface	Contrato implementado por todos os canais (Observer)
NotificacaoEmail / SMS / WhatsApp / Interna	Classe	Observadores concretos — um por canal
GerenciadorDeNotificacoes	Classe	Subject — gerencia e notifica os observadores
AluguelService	Classe (modificada)	Publisher — dispara os eventos

Como consequência, a inclusão de um novo canal — por exemplo, push notification — exige apenas a criação de uma nova classe que implemente `Notificavel` e seu registro no `GerenciadorDeNotificacoes`, sem alteração de código existente.

2.2 Montagem de Pacotes de Hospedagem — Padrão Decorator

O sistema permite compor livremente serviços adicionais à hospedagem — café da manhã (R\$ 35,00), transporte (R\$ 60,00), lavanderia (R\$ 45,00) e passeio turístico (R\$ 90,00) —, com o preço total calculado dinamicamente conforme os itens selecionados.

Uma solução baseada em herança exigiria uma classe para cada combinação possível de serviços, resultando em explosão combinatória. Optou-se, portanto, pelo padrão Decorator, que envolve dinamicamente um objeto base (`HospedagemBase`) com decoradores de serviço, cada um responsável por agregar sua própria descrição e preço ao objeto decorado.

Elemento	Tipo	Papel no padrão
ServicoPacote	Interface	Componente — contrato com <code>getDescricao()</code> e <code>getPreco()</code>
HospedagemBase	Classe	Componente concreto — objeto base a ser decorado
ServicoDecorator	Classe abstrata	Decorator abstrato — envolve um <code>ServicoPacote</code>
CafeDaManha / Transporte / Lavanderia / PasseioTuristico	Classe	Decoradores concretos — um por serviço

A inclusão de um novo serviço, como traslado ao aeroporto, exige apenas a criação de uma nova classe que estenda `ServicoDecorator`, preservando o restante do código inalterado.

3. Padrão Singleton

O `GerenciadorDeNotificacoes` foi implementado como Singleton, garantindo uma única instância da central de notificações em todo o sistema. Essa decisão evita inconsistências decorrentes de múltiplas instâncias — como canais registrados em objetos distintos, o que poderia causar notificações duplicadas ou perdidas.

A implementação segue a estrutura clássica do padrão: construtor privado, que impede a instanciamento externa; atributo estático `instancia`, responsável por armazenar a única instância da classe; e método estático `getInstancia()`, que retorna a instância existente ou a cria caso ainda não exista. O `AluguelService` obtém essa instância por meio de `GerenciadorDeNotificacoes.getInstancia()` no método anotado com `@PostConstruct`, registrando os canais uma única vez durante a inicialização do serviço.

4. Justificativa das Escolhas

- Observer resolve a comunicação entre componentes sem acoplamento direto, adequando-se a um cenário de notificações que precisa escalar para novos canais.
- Decorator resolve a composição dinâmica de funcionalidades, adequando-se à necessidade de combinações livres de serviços sem explosão de classes.
- Ambos os padrões aderem ao princípio Aberto/Fechado (OCP): permitem extensão sem exigir modificação do código existente.
- Trata-se de padrões consolidados na literatura (GoF), com aplicação direta ao domínio do problema, o que favorece a legibilidade e a manutenção do sistema.

5. Conclusão

A Sprint 4 consolidou a evolução arquitetural do Sistema de Hospedagem por meio da aplicação de padrões de projeto voltados à solução de problemas concretos identificados nas sprints anteriores. A Central de Notificações, estruturada com o padrão Observer, e a Montagem de Pacotes, estruturada com o padrão Decorator, somadas ao uso do Singleton no `GerenciadorDeNotificacoes`, evidenciam a capacidade do sistema de evoluir de forma organizada, extensível e alinhada às boas práticas da Engenharia de Software.